



دانشکده مهندسی کامپیوتر
سیستم های عامل

نیمسال اول ۱۴۰۵-۱۴۰۴
۴۰۲۱۰۶۵۴۲، ۴۰۲۱۰۸۷۶

حسین اسدی
علی مقدسی، علیرضا سرباز

تمرین دوم

۱ سوال ۳: Fork Bomb

۱.۱ مفهوم Fork Bomb و نحوه عملکرد آن

Fork Bomb یک نوع حمله Denial of Service است که با ایجاد تعداد بسیار زیادی پردازش در مدت زمان کوتاه، منابع سیستم (یعنی CPU، حافظه و جدول پردازشها) را مصرف می کند و سیستم را غیرقابل استفاده می سازد.

نحوه عملکرد Fork Bomb معمولاً یک حلقه بی نهایت است که در هر تکرار، با استفاده از فراخوان سیستمی `fork()`، یک کپی از خودش ایجاد می کند. هر پردازش جدید نیز همین کار را انجام می دهد، بنابراین تعداد پردازشها به صورت نمایی رشد می کند. برای مثال، اگر در هر ثانیه هر پردازش یک پردازش جدید ایجاد کند، پس از ۱۰ ثانیه بیش از ۱۰۰۰ پردازش خواهیم داشت، و پس از ۲۰ ثانیه بیش از یک میلیون پردازش.

۲.۱ نمونه ساده Fork Bomb

یک نمونه ساده Fork Bomb در فایل `fork_bomb.c` پیاده سازی شده است:

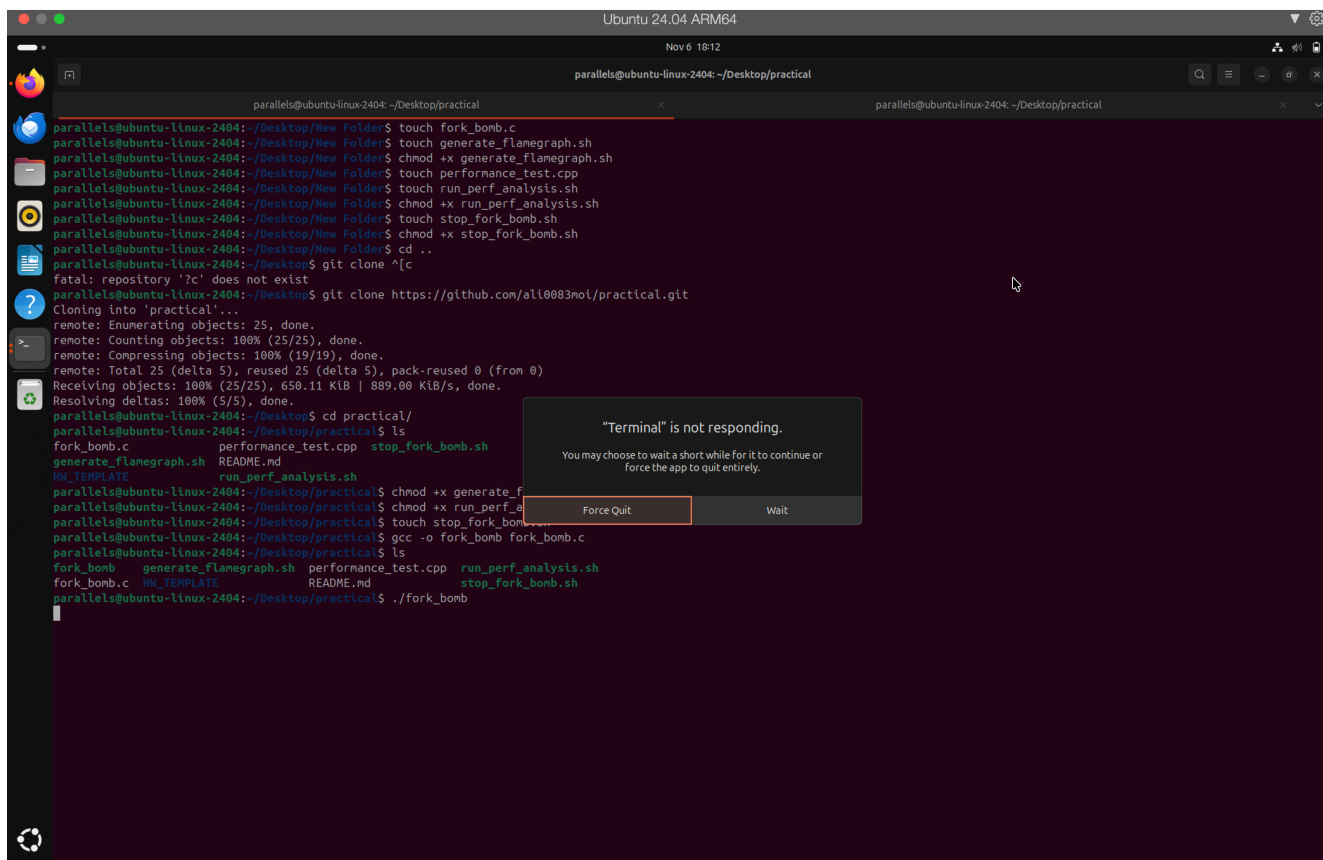
```
#include <unistd.h>
#include <stdio.h>
#include <sys/types.h>

int main() {
    while(1) {
        fork();
    }
    return 0;
}
```

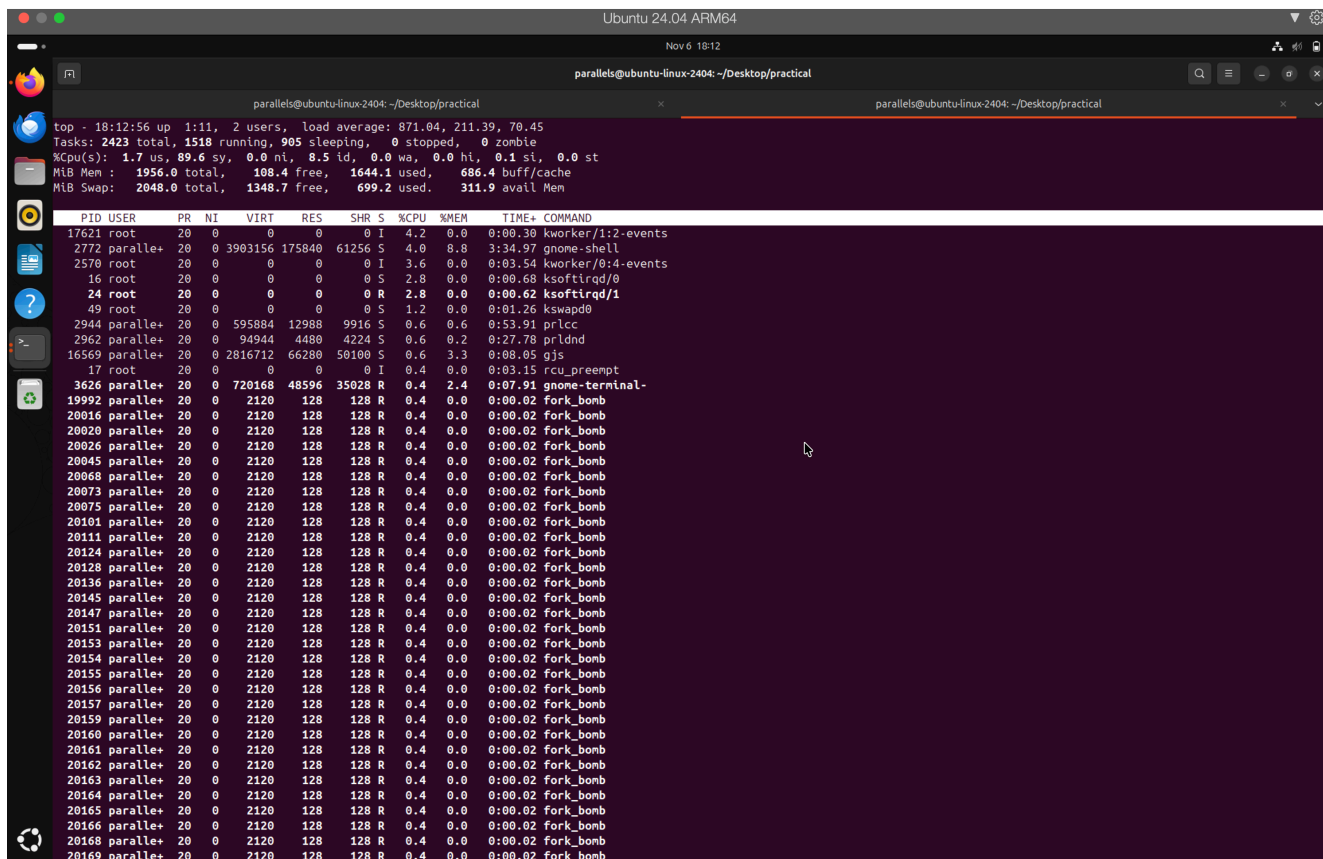
برای کامپایل و اجرا:

```
gcc -o fork_bomb fork_bomb.c
./fork_bomb
```

نتایج اجرا:



در این عکس مشاهده می شود که بعد از اجرای کد fork_bomb.c، تعداد پردازنده ها به صورت نمایی رشد می کند و ترمینال از کار می افتد.



در این عکس به کمک دستور top که از قبل باز بود می‌توانیم مشاهده کنیم که تعداد خیلی زیادی fork و پراسس برای دستور fork_bomb.c ایجاد شده است.

```
ubuntu-linux-2404 login: ali0083moi
Password:

Login incorrect
ubuntu-linux-2404 login: parallels
Password:
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-51-generic aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Nov  6 06:08:00 PM +0330 2025

System load:          0.21
Usage of /:            19.9% of 61.66GB
Memory usage:         53%
Swap usage:           23%
Processes:            216
Users logged in:      1
IPv4 address for enp0s5: 10.211.55.3
IPv6 address for enp0s5: fdb2:2c26:f4e4:0:5a5a:ce3:e037:3824
IPv6 address for enp0s5: fdb2:2c26:f4e4:0:4815:c300:ba88:6e4f

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

584 updates can be applied immediately.
253 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

parallels@ubuntu-linux-2404:~$ pskill -9 fork_bomb

^C
parallels@ubuntu-linux-2404:~$ sudo killall -9 fork_bomb
[sudo] password for parallels:
parallels@ubuntu-linux-2404:~$
```

برای اینکه بتوانیم fork_bomb را متوقف کنیم به کمک زدن کلید های ctrl+alt+f3 وارد یک ترمینال مجازی می‌شویم که در آن می‌توانیم به کمک زدن دستور killall lrsudo fork_bomb را متوقف کنیم.

```

top - 18:14:25 up 1:12, 3 users, load average: 1423.70, 623.55, 232.85
Tasks: 266 total, 1 running, 265 sleeping, 0 stopped, 0 zombie
%Cpu(s): 17.3 us, 5.9 sy, 0.0 ni, 76.5 id, 0.2 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 1956.0 total, 126.5 free, 1504.8 used, 855.9 buff/cache
MiB Swap: 2048.0 total, 1317.0 free, 731.0 used, 451.2 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
 2772 paralle+  20   0 3918256 189428 73780 S 16.6  9.5   3:36.14 gnome-shell
 3626 paralle+  20   0 728720 48596 34900 S  5.0  2.4   0:08.12 gnome-terminal-
 1155 polkitd    20   0 389544  9216  5760 S  3.3  0.5   0:03.71 polkitd
 9927 paralle+  20   0 3285372 153716 71672 S  2.6  7.7   1:01.81 firefox
1305074 gdm        20   0 3692336 192660 107328 S  2.3  9.6   0:01.15 gnome-shell
 2944 paralle+  20   0 595884 12988  9916 S  1.7  0.6   0:54.01 prlcc
 1145 message+  20   0 12468  6016  3584 S  1.3  0.3   0:18.73 dbus-daemon
 2592 paralle+  9  -11 409672 15424 11328 S  1.0  0.8   0:00.28 wireplumber
10682 paralle+  20   0 2631936 167252 43188 S  1.0  8.4   0:15.31 Isolated Web Co
   1 root      20   0 23224 10060  6476 S  0.7  0.5   0:01.95 systemd
 2603 paralle+  20   0 11208  5632  3712 S  0.7  0.3   0:10.77 dbus-daemon
16569 paralle+  20   0 2816712 63256 47028 S  0.7  3.2   0:08.42 gjs
1305005 root       20   0 0 0 0 I  0.7  0.0   0:00.02 kworker/0:0-events
1305025 gdm        9  -11 407624 17772 12652 S  0.7  0.9   0:00.06 wireplumber
 549 root      19  -1  67208 11820 11052 S  0.3  0.6   0:00.71 systemd-journal
 623 root      20   0 30684  3840  2944 S  0.3  0.2   0:00.95 systemd-udev
1158 root      20   0 311712 4992  4736 S  0.3  0.2   0:00.06 power-profiles-
1195 root      20   0 308076 4992  4480 S  0.3  0.2   0:00.02 switcheroo-cont
 2913 paralle+  20   0 602224 12676 10244 S  0.3  0.6   0:00.19 gsd-power
 2940 paralle+  20   0 900936 13244 11068 S  0.3  0.7   0:00.18 evolution-alarm
 2962 paralle+  20   0 94944  4480  4224 S  0.3  0.2   0:27.81 prldad
 3091 paralle+  20   0 425300 15092  8280 S  0.3  0.8   0:01.44 ibus-extension-
 3320 paralle+  20   0 713484 14536 11900 S  0.3  0.7   0:00.32 xdg-desktop-por
 3332 paralle+  20   0 2527232 13724 11264 S  0.3  0.7   0:00.12 gjs
 3442 paralle+  20   0 421336 11388  8828 S  0.3  0.6   0:00.12 xdg-desktop-por
19053 paralle+  20   0 14116  7552 3200 R  0.3  0.4   0:00.30 top
1305023 gdm        9  -11 125504 11796  7828 S  0.3  0.6   0:00.01 pipewire
1305425 gdm        20   0 387048 11612  6528 S  0.3  0.6   0:00.05 ibus-daemon
1305488 gdm        20   0 234856  6784  6272 S  0.3  0.3   0:00.01 ibus-engine-sim
   2 root      20   0 0 0 0 S  0.0  0.0   0:00.00 kthread
   3 root      20   0 0 0 0 S  0.0  0.0   0:00.00 pool_workqueue_release
   4 root      0 -20 0 0 0 I  0.0  0.0   0:00.00 kworker/R-rcu_g
   5 root      0 -20 0 0 0 I  0.0  0.0   0:00.00 kworker/R-rcu_p
   6 root      0 -20 0 0 0 I  0.0  0.0   0:00.00 kworker/R-slab_
   7 root      0 -20 0 0 0 I  0.0  0.0   0:00.00 kworker/R-netns
  10 root      0 -20 0 0 0 I  0.0  0.0   0:00.00 kworker/0:0H-kblockd
  12 root      0 -20 0 0 0 I  0.0  0.0   0:00.00 kworker/R-mm_pg
  13 root      20   0 0 0 0 I  0.0  0.0   0:00.00 rcu_tasks_kthread
  14 root      20   0 0 0 0 I  0.0  0.0   0:00.00 rcu_tasks_rude_kthread
  15 root      20   0 0 0 0 I  0.0  0.0   0:00.00 rcu_tasks_trace_kthread
  16 root      20   0 0 0 0 S  0.0  0.0   0:05.31 ksoftirqd/0
  17 root      20   0 0 0 0 I  0.0  0.0   0:03.69 rcu_preempt

```

همان طور که در عکس مشخص است بعد از kill کردن این پراسس ها دیگر آن ها در top دیده نمی شوند.

```

parallels@ubuntu-linux-2404: ~/Desktop/practical
parallels@ubuntu-linux-2404:~/Desktop/practical$ touch fork_bomb.c
parallels@ubuntu-linux-2404:~/Desktop/practical$ touch generate_flamegraph.sh
parallels@ubuntu-linux-2404:~/Desktop/practical$ chmod +x generate_flamegraph.sh
parallels@ubuntu-linux-2404:~/Desktop/practical$ touch performance_test.cpp
parallels@ubuntu-linux-2404:~/Desktop/practical$ touch run_perf_analysis.sh
parallels@ubuntu-linux-2404:~/Desktop/practical$ chmod +x run_perf_analysis.sh
parallels@ubuntu-linux-2404:~/Desktop/practical$ touch stop_fork_bomb.sh
parallels@ubuntu-linux-2404:~/Desktop/practical$ chmod +x stop_fork_bomb.sh
parallels@ubuntu-linux-2404:~/Desktop/practical$ cd ..
parallels@ubuntu-linux-2404:~/Desktop$ git clone ^[c
fatal: repository 'c' does not exist
parallels@ubuntu-linux-2404:~/Desktop$ git clone https://github.com/al10083moi/practical.git
Cloning into 'practical'...
remote: Enumerating objects: 25, done.
remote: Counting objects: 100% (25/25), done.
remote: Compressing objects: 100% (19/19), done.
remote: Total 25 (delta 5), reused 25 (delta 5), pack-reused 0 (from 0)
Receiving objects: 100% (25/25), 650.11 KiB | 889.00 KiB/s, done.
Resolving deltas: 100% (5/5), done.
parallels@ubuntu-linux-2404:~/Desktop$ cd practical/
parallels@ubuntu-linux-2404:~/Desktop/practical$ ls
fork_bomb.c      performance_test.cpp  stop_fork_bomb.sh
generate_flamegraph.sh  README.md
HW_TEMPLATE      run_perf_analysis.sh
parallels@ubuntu-linux-2404:~/Desktop/practical$ chmod +x generate_flamegraph.sh
parallels@ubuntu-linux-2404:~/Desktop/practical$ chmod +x run_perf_analysis.sh
parallels@ubuntu-linux-2404:~/Desktop/practical$ touch stop_fork_bomb.sh
parallels@ubuntu-linux-2404:~/Desktop/practical$ gcc -o fork_bomb fork_bomb.c
parallels@ubuntu-linux-2404:~/Desktop/practical$ ls
fork_bomb      generate_flamegraph.sh  performance_test.cpp  run_perf_analysis.sh
fork_bomb.c    HW_TEMPLATE            README.md             stop_fork_bomb.sh
parallels@ubuntu-linux-2404:~/Desktop/practical$ ./fork_bomb
Killed
parallels@ubuntu-linux-2404:~/Desktop/practical$

```

در این عکس هم مشخص است که fork_bomb کیل شده است.

۳.۱ اثرات Bomb Fork روی منابع سیستم

برای تحلیل اثرات Bomb Fork روی منابع سیستم، می‌توان از دستورات زیر استفاده کرد:

```
# View number of processes
ps aux | wc -l

# View CPU and memory usage
top
# or
htop

# View system limits
ulimit -a
```

با کمک `ctrl+c` این مشکل حل نمی‌شود زیرا این دستور برای استاپ کردن یک پراسس است اما `fork_bomb` تعداد خیلی زیادی پراسس ایجاد می‌کند که نمی‌توان به کمک این دستور آن‌ها را استاپ کرد. برای جلوگیری از این مشکل می‌توان از دستور `ulimit -u 100` استفاده کرد تا برای `fork` های یوزر лимیت گذاشت و نگذاریم که پراسس‌ها از یک تعدادی بیشتر شوند.

۴.۱ راه‌های مقابله با Bomb Fork

راه‌های مختلفی برای مقابله با Bomb Fork وجود دارد:

۱. استفاده از **ulimit**: می‌توان تعداد پردازش‌های مجاز برای یک کاربر را محدود کرد:

```
ulimit -u 100 # Limit to 100 processes
```

۲. استفاده از **cgroups**: در سیستم‌های مدرن لینوکس، می‌توان از `cgroups` برای محدود کردن منابع استفاده کرد.

۳. کشتن پردازش‌ها: استفاده از `pkill` یا `killall` برای متوقف کردن تمام پردازش‌های Bomb Fork:

```
pkill 9- fork_bomb
# or
killall 9- fork_bomb
```

۴. استفاده از اسکریپت خودکار: اسکریپت `stop_fork_bomb.sh` برای این منظور تهیه شده است.

۵. تنظیمات سیستم: می‌توان در فایل `/etc/security/limits.conf` محدودیت‌های دائمی تعریف کرد.

۵.۱ اجرای fork bomb و مهار کردن آن

در بخش ۱.۲ کامل توضیح داده شده است.

۲ سوال ۴: نمایش و تحلیل کارایی برنامه

۱.۲ ابزار perf در لینوکس

perf یک ابزار قدرتمند برای تحلیل عملکرد در لینوکس است که امکان جمع‌آوری داده‌های عملکردی در سطح هسته سیستم‌عامل و سخت‌افزار را فراهم می‌سازد.

ویژگی‌های اصلی perf:

- نمونه‌برداری از CPU: جمع‌آوری اطلاعات درباره توابعی که بیشترین زمان CPU را مصرف می‌کنند
 - تحلیل رویدادهای سخت‌افزاری: دسترسی به Performance Monitoring Unit (PMU) برای تحلیل cache misses، branch mispredictions و غیره
 - تحلیل call stack: امکان مشاهده زنجیره فراخوانی توابع
 - تحلیل زمان واقعی: امکان مشاهده عملکرد برنامه در حین اجرا
- دستورات مفید perf :

```
# Record samples
perf record -g ./program

# Display report
perf report

# Interactive display
perf top

# Save output for FlameGraph
perf script > perf_script.out
```

۲.۲ برنامه C++ با بخش‌های نابینه

یک برنامه C++ پیچیده با بخش‌های نابینه در فایل performance_test.cpp نوشته شده است که شامل موارد زیر است:

- محاسبه فاکتوریل به صورت بازگشتی بدون بهینه‌سازی
- جستجوی خطی در آرایه مرتب نشده
- مرتب‌سازی حبابی (Bubble Sort)
- محاسبات ریاضی پیچیده و تکراری
- کپی گرفتن از داده‌ها

این برنامه به گونه‌ای طراحی شده که حداقل ۱۰ ثانیه اجرای آن طول بکشد تا امکان تحلیل دقیق‌تر فراهم شود.

۳.۲ تحلیل عملکرد با perf

برای تحلیل عملکرد برنامه با perf، از اسکریپت run_perf_analysis.sh استفاده می‌شود: این اسکریپت برنامه را به کمک فلگ -O0 -g کامپایل می‌کند و سپس با perf record نمونه‌برداری انجام می‌دهد و سپس گزارش perf را تولید می‌کند و خروجی را برای FlameGraph ذخیره می‌کند. بعد از اجرا کردن آن خروجی در فایل‌های perf_report.txt و perf_script.out ثبت شده است.

۴.۲ FlameGraph

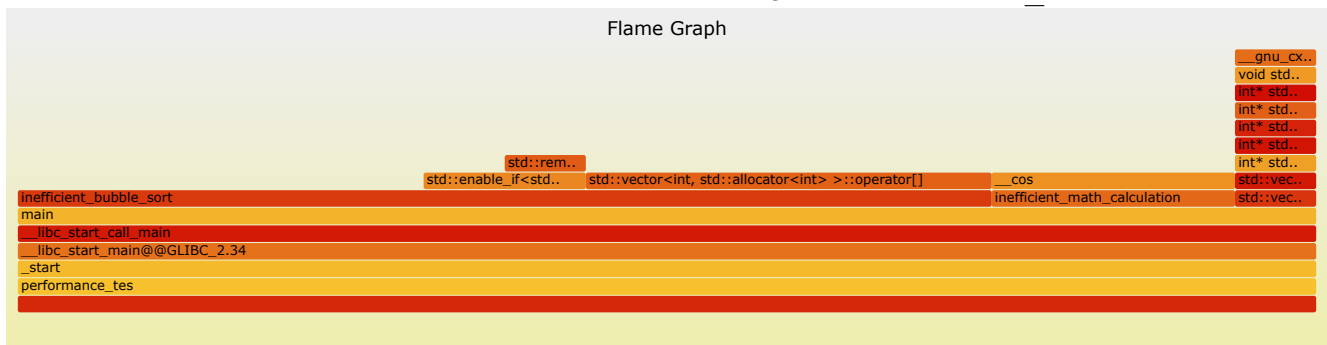
FlameGraph یک ابزار تجسم برای نمایش داده‌های عملکردی به صورت گرافیکی است. این ابزار داده‌های stack call را به صورت یک گراف افقی نمایش می‌دهد که در آن:

- عرض هر مستطیل نشان‌دهنده زمان صرف شده در آن تابع است
- نشان‌دهنده عمق stack call است
- رنگ‌ها برای تمایز بین توابع مختلف استفاده می‌شوند

به کمک اسکریپت generate_flamegraph.sh می‌توان FlameGraph را تولید کرد:

```
chmod +x generate_flamegraph.sh
./generate_flamegraph.sh
```

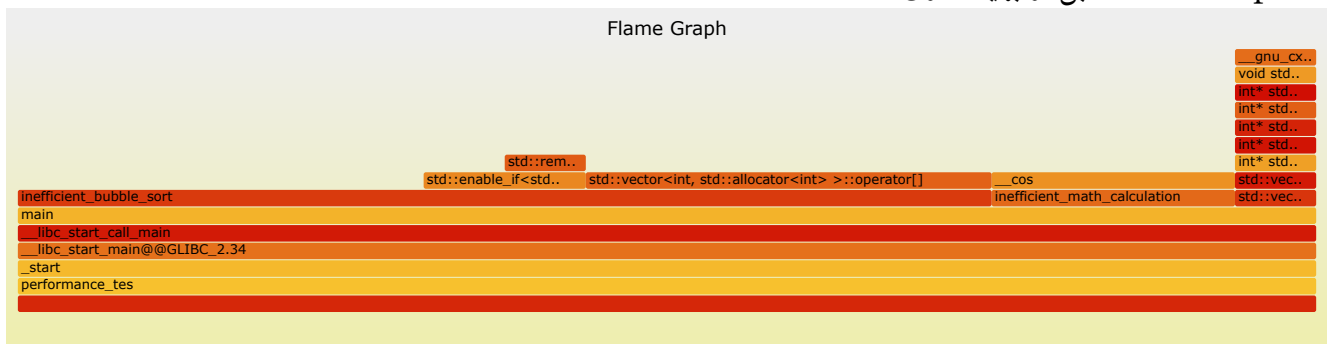
خروجی flamegraph_before.svg در فایل‌های موجود است. که به صورت زیر است:



۵.۲ تولید FlameGraph از خروجی perf

با استفاده از خروجی perf script می‌توان یک FlameGraph تولید کرد که بخش‌های نابهنه برنامه را به وضوح نشان می‌دهد.

FlameGraph قبل از بهینه‌سازی:



تحلیل FlameGraph و شناسایی بخش‌های نابهنه:

با بررسی FlameGraph قبل از بهینه‌سازی، بخش‌های زیر به عنوان گلوگاه‌های عملکردی شناسایی شدند:

- تابع inefficient_factorial: این تابع به صورت بازگشتی پیاده‌سازی شده و برای هر فراخوانی، چندین بار فراخوانی بازگشتی انجام می‌دهد که باعث سربار زیاد می‌شود.
- تابع inefficient_search: جستجوی خطی در آرایه مرتب نشده با پیچیدگی زمانی $O(n)$ برای هر جستجو انجام می‌شود که برای ۱۰۰۰ جستجو در آرایه ۱۰۰۰۰ عنصری بسیار ناکارآمد است.

- تابع `inefficient_bubble_sort`: مرتب‌سازی حبابی با پیچیدگی زمانی $O(n^2)$ یکی از کندترین الگوریتم‌های مرتب‌سازی است.
- تابع `inefficient_math_calculation`: محاسبات ریاضی پیچیده (سینوس، کسینوس، جذر) در هر تکرار حلقه انجام می‌شود که بسیار هزینه‌بر است.
- تابع `inefficient_copy_operations`: این تابع ۴ بار کپی غیرضروری از داده انجام می‌دهد که باعث مصرف زیاد حافظه و زمان می‌شود.

۶.۲ بهینه‌سازی و بهبود

بر اساس تحلیل FlameGraph، بخش‌های ناهینه شناسایی و بهبود داده شدند:

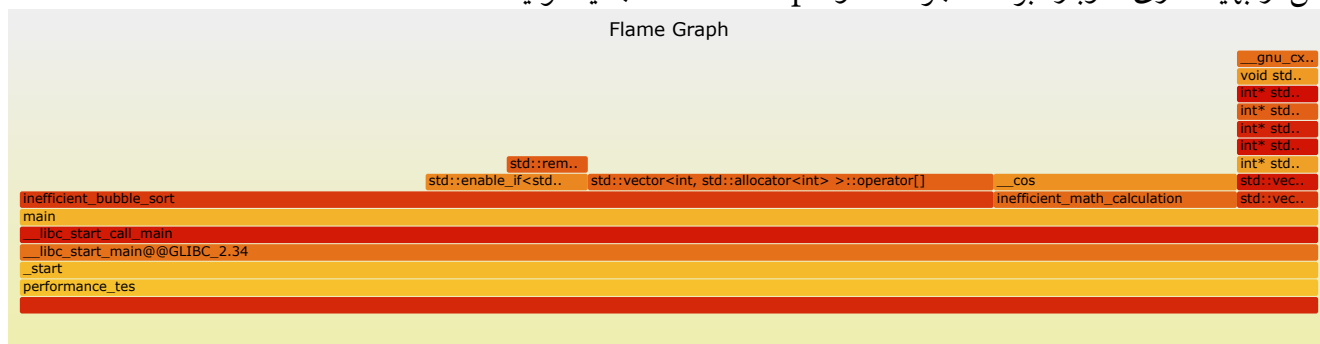
- جایگزینی فاکتوریل بازگشتی با نسخه `iterative` یا استفاده از `table lookup`
- استفاده از جستجوی دودویی به جای جستجوی خطی (پس از مرتب‌سازی)
- جایگزینی مرتب‌سازی حبابی با الگوریتم‌های سریع‌تر مثل `Sort Quick` یا `Sort Merge`
- بهینه‌سازی محاسبات ریاضی با کاهش تکرارها و استفاده از `tables lookup`
- حذف کپی‌های غیرضروری با استفاده از `reference` یا `semantics move`

کد بهینه‌شده در فایل `performance_test_optimized.cpp` پیاده‌سازی شده است. تغییرات اصلی:

۱. فاکتوریل بهینه‌شده: استفاده از محاسبه `iterative` به جای بازگشتی و اضافه کردن `cache` برای جلوگیری از محاسبات تکراری.
۲. جستجوی دودویی: پس از مرتب‌سازی یک‌باره آرایه، از جستجوی دودویی با پیچیدگی $O(\log n)$ استفاده می‌شود.
۳. مرتب‌سازی بهینه: استفاده از `std::sort` که از الگوریتم `introsort` (ترکیبی از `quicksort`، `heapsort` و `insertion sort`) استفاده می‌کند.
۴. محاسبات ریاضی بهینه: پیش‌محاسبه مقادیر سینوس و کسینوس در آرایه‌ها و استفاده از آن‌ها به جای محاسبه در هر تکرار.
۵. حذف کپی‌های غیرضروری: استفاده از `reference` به جای کپی برای جلوگیری از سربار حافظه.

۷.۲ نمایش نتایج بهینه‌سازی در FlameGraph

پس از بهینه‌سازی، دوباره برنامه اجرا شده و FlameGraph جدید تولید شد:



۸.۲ ابزار tracey

tracey یک تحلیل‌گر عملکرد بلادرنگ است که امکان مشاهده رفتار برنامه در حین اجرا را فراهم می‌کند.
ویژگی‌های tracey:

- تحلیل بلادرنگ عملکرد
- نمایش فراخوان‌های سیستمی
- تحلیل I/O operations
- نمایش activity network

نصب و استفاده: RESULT_TODO

۹.۲ تحلیل برنامه با tracey

برای تحلیل برنامه با tracey:

```
tracey ./performance_test
```

نتایج تحلیل: tracey RESULT_TODO